Methodology

Data Preparation

I used PySpark to handle the initial data processing. I sampled the Wikipedia dataset to get 1172 documents. One challenge I encountered was probably a race condition when writing files to the local disk and immediately reading them back into an RDD. I constantly got 1100 files in my HDFS instead of the whole 1172. To solve this, I modified the pipeline to map the DataFrame directly to an RDD in memory before saving to HDFS.

Indexer Logic (Hadoop MapReduce)

I implemented a single-pass MapReduce pipeline to build the inverted index.

- Mapper tokenizes the text into lowercase alphabetic words and emits two types of data: Metadata (tagged with `META`) for corpus statistics and Term Frequencies for the index.
- Reducer takes advantage of Hadoop's internal sorting. Because `META` sorts alphabetically before word tokens (uppercase), I was able to calculate global stats (N and avg_dl) before processing the actual vocabulary. This allowed me to output all necessary statistics for the BM25 formula in one single job.

Storage

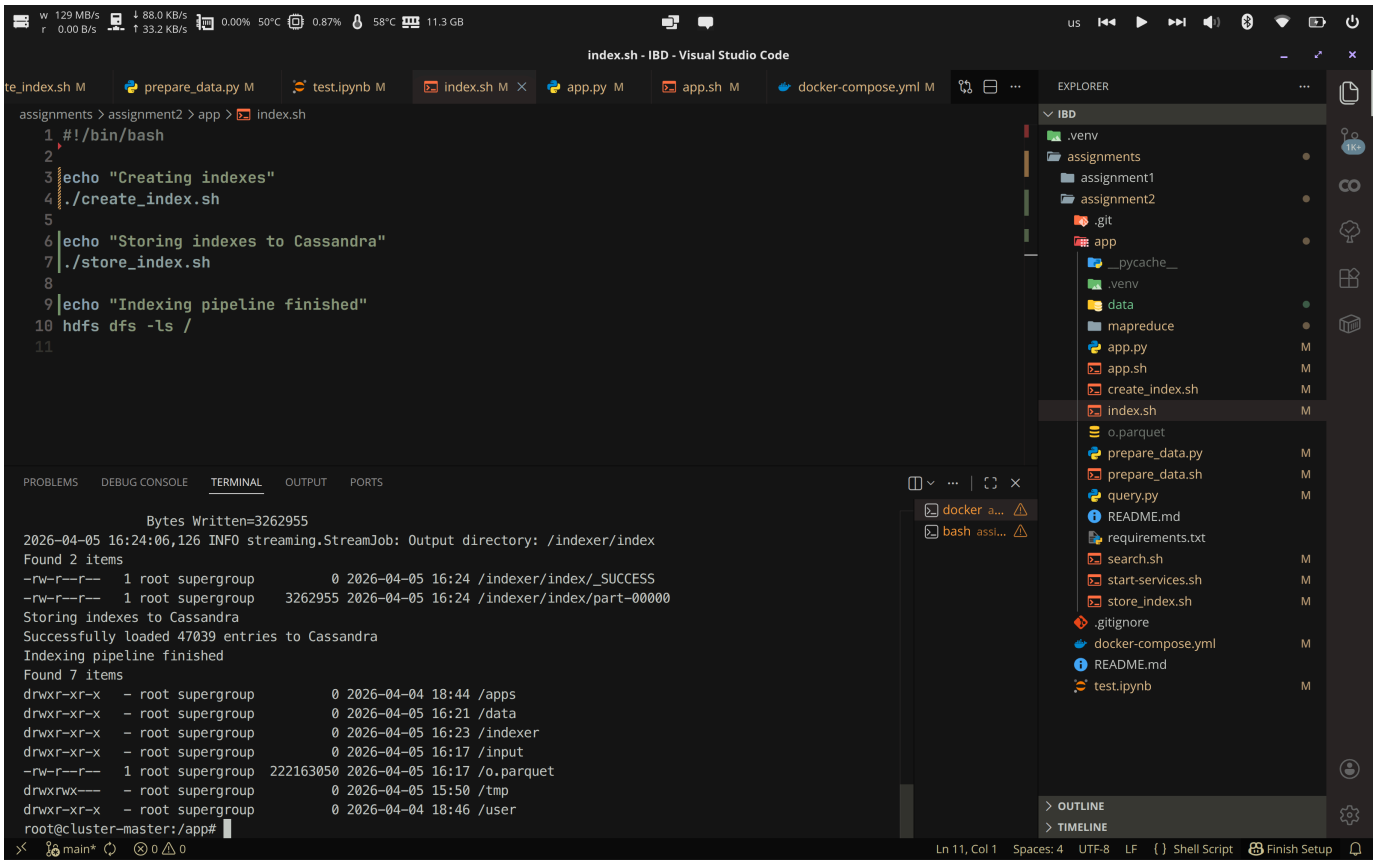
I used the `cassandra-driver` in Python to move the index from HDFS to a database. I created three tables: `corpus_stats`, `doc_lens`, and `inverted_index`. To prevent memory issues, I used `subprocess.Popen` to stream HDFS data line-by-line and used a prepared statements for faster insertions.

Ranking (BM25)

The ranker was built using the PySpark RDDs. I used broadcast variables to send the corpus statistics to every worker node, preventing unnecessary database lookups. The search engine calculates BM25 scores for every document containing the query terms, sums them up, and retrieves top 10 results.

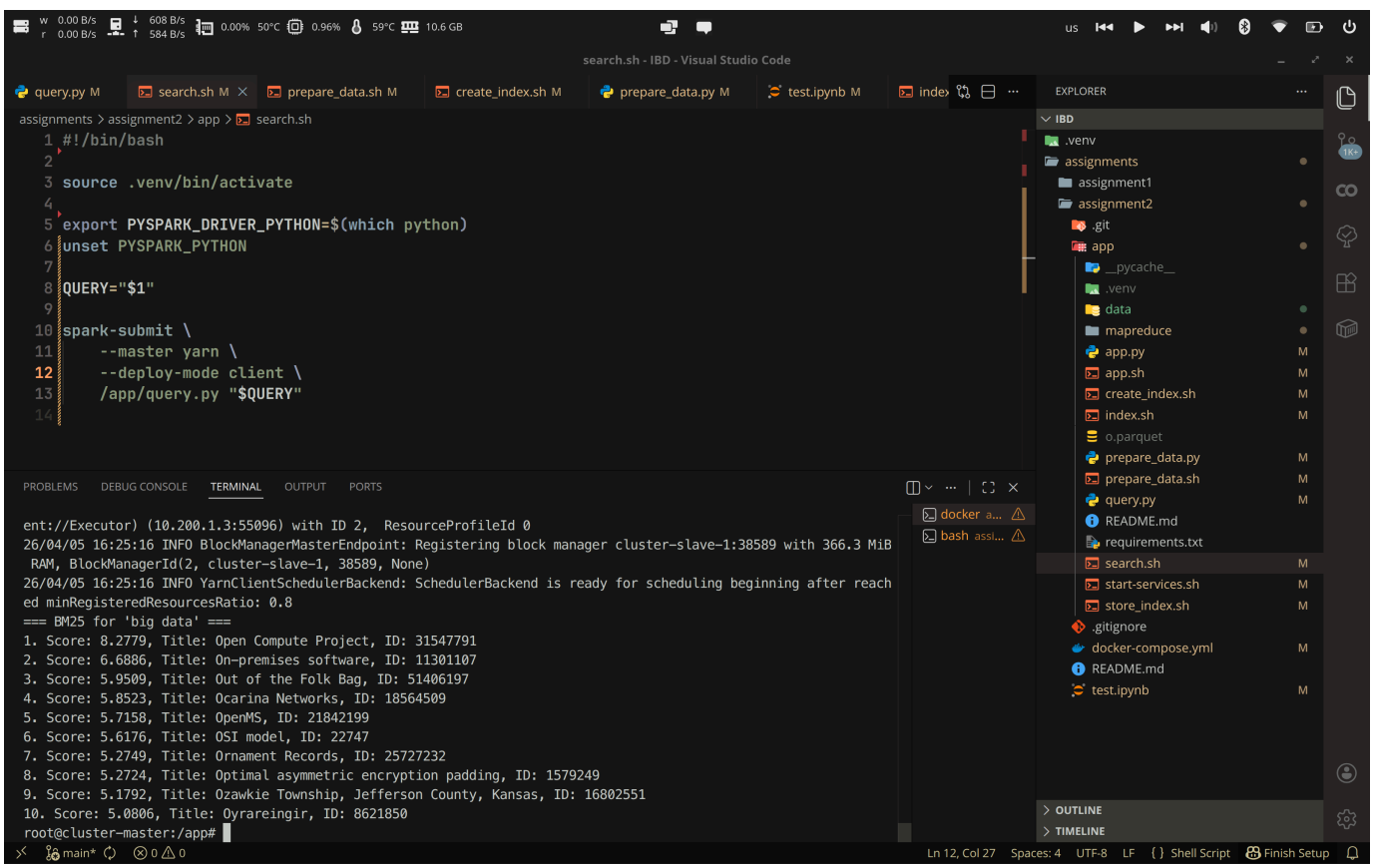
Demonstration

Indexing Execution



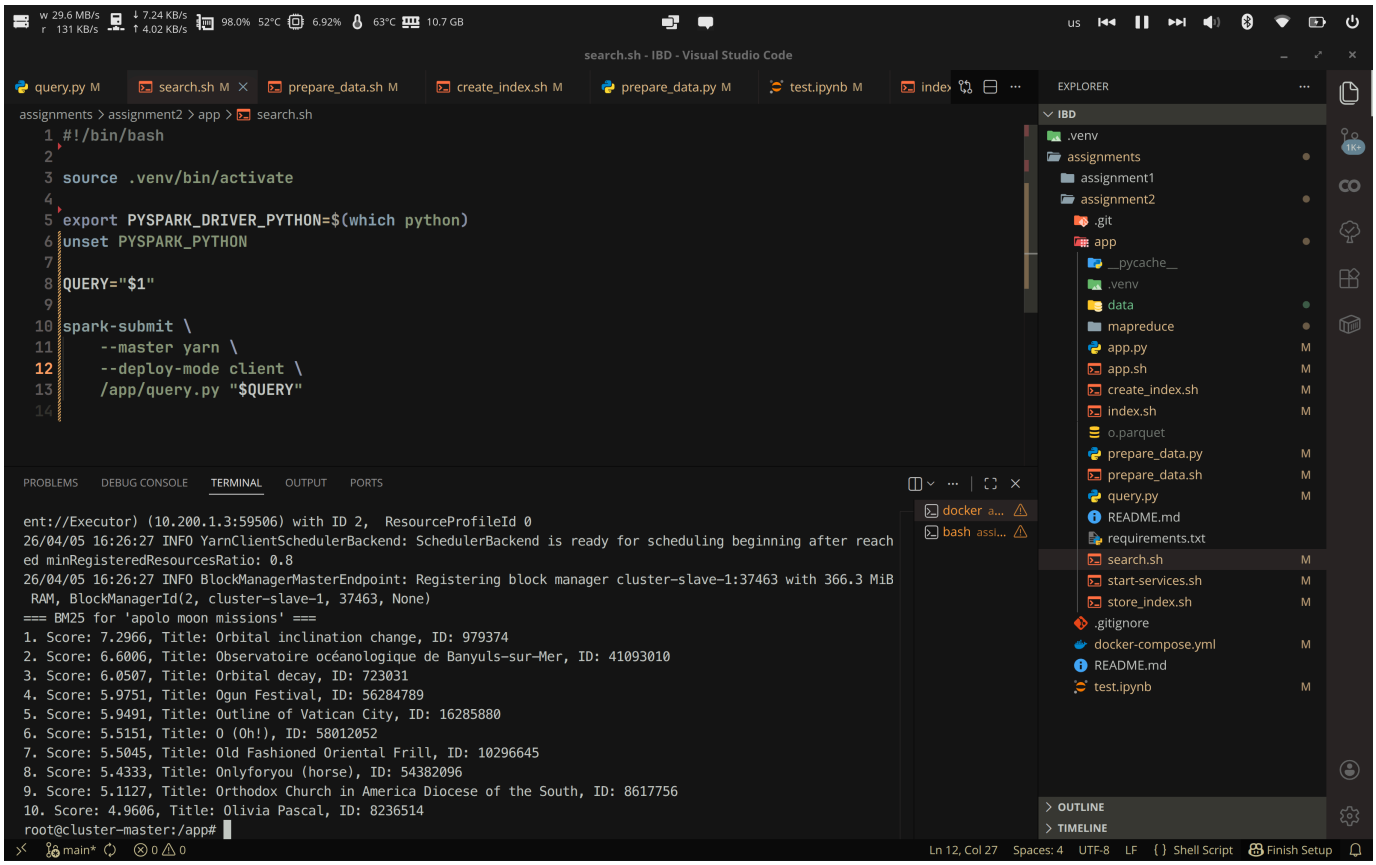
In this step, the MapReduce job processed 1172 records. The final load into Cassandra resulted in 47039 entries.

Query 1: "big data"



The search for "big data" returned data-related files such as "Open Compute Project" and "OSI model". BM25 algorithm successfully prioritized documents where these searched terms appeared frequently relative to the document length compared to other documents.

Query 2: "apolo moon missions"



The system retrieved documents related to orbital phenomenons and space. Looking back at it, I actually misspelled Apollo, so the results being space related are due to the "moon" part. The misspelled "apolo" was likely not even used in scoring because it provided no matches - BM25 is a keyword matching algorithm.

Reflections

Building this search engine showed me the practical trade offs between HDFS and NoSQL databases. While MapReduce is great for batch processing the index, querying HDFS directly for a search would be too slow ($O(N)$). By moving the index to Cassandra, lookups became nearly instantaneous ($O(1)$). Working on a 16GB machine required careful management of container resources, but the final pipeline is robust and handles document length normalization much better than a standard TF-IDF.